

Arduino Distance Meter — Complete Knowledge Base

Using HC-SR04 Ultrasonic Sensor + LCD Display + Laser Pointer

1. Introduction — What Are We Building?

Imagine a superhero gadget that can tell you exactly how far away something is — just like a bat using sound to navigate in the dark! That is exactly what this project does.

We built a **Distance Meter** using an Arduino microcontroller. You point it at something, and a small screen tells you how far away that object is in centimetres. A red laser dot lights up what you are measuring, just like a real rangefinder!

Components Used

Component	Model	Role
Microcontroller	Arduino Uno (ATmega328P)	The brain — runs all the code
Distance Sensor	HC-SR04 Ultrasonic	Measures distance using sound waves
Display	16x2 LCD with I2C module	Shows the distance reading on screen
Laser Emitter	HW-493 (KY-008)	Red dot pointer — shows what is being measured

2. How Each Component Works

2.1 Arduino Uno — The Brain

Think of the Arduino like a tiny computer that only does ONE job, really really well. It reads information from sensors, does some math, and tells other parts what to do.

The Arduino Uno uses a chip called **ATmega328P**. It runs at 16 MHz — meaning it can do 16 million tiny operations per second. It has:

- **14 Digital pins** — can be ON (5V) or OFF (0V)
 - **6 Analog pins** — can read values between 0V and 5V (like a dimmer switch)
 - **2KB of RAM** — temporary memory while running
 - **32KB of Flash** — permanent memory that stores your code
-

2.2 HC-SR04 Ultrasonic Sensor — The Ears

The HC-SR04 measures distance the same way a bat does — using sound! Those **two circles** on the front are the transmitter (sends sound) and receiver (listens for the echo).

How it works step by step:

1. Arduino sends a tiny 10-microsecond HIGH pulse to the TRIG pin
2. The sensor fires **8 bursts of ultrasonic sound** at 40,000 Hz (too high for humans to hear)
3. Sound travels through air, hits an object, and bounces back
4. The sensor detects the returning echo and sends a pulse on the ECHO pin
5. Arduino measures how long the ECHO pin was HIGH — that is the travel time

The Distance Formula:

Speed of sound in air = 343 metres/second = 0.0343 cm/microsecond

The sound travels TO the object AND BACK, so we divide by 2:

$$1 \quad \text{Distance (cm)} = (\text{Duration in microseconds} \times 0.0343) \div 2$$

Example: If echo duration = 1166 μs $\rightarrow$ Distance = $(1166 \times 0.0343) / 2 = 20 \text{ cm}$

Specifications:

Spec	Value
Operating Voltage	5V DC
Measuring Range	2 cm to 400 cm
Accuracy	$\pm 3 \text{ mm}$
Trigger Pulse	10 μs HIGH signal
Ultrasonic Frequency	40,000 Hz

2.3 LCD 16x2 with I2C Module — The Screen

LCD stands for **Liquid Crystal Display**. The 16x2 means it can show **16 characters per row** across **2 rows** — 32 characters total.

The I2C module (the small board on the back of the LCD) is a communication helper. Without it, you would need 10+ wires to control the LCD. With I2C, you only need **4 wires!**

What is I2C?

I2C (Inter-Integrated Circuit) is a communication protocol — a language devices use to talk to each other. It uses only two signal wires:

- **SDA (Serial Data)** — carries the actual data
- **SCL (Serial Clock)** — keeps both sides in sync, like a metronome

Each I2C device has a unique address (like a house number). Our LCD lives at address **0x27**. The Arduino sends data to that address and only the LCD responds.

There is also a small **blue potentiometer** on the I2C module — turning it adjusts LCD contrast (how dark the characters appear).

2.4 HW-493 Laser Module (KY-008) — The Pointer

The HW-493 is a simple **red laser diode** on a PCB. It emits a 650nm wavelength red laser dot at 5mW power — same as a laser pen.

It has 3 pins: Signal (S), VCC (5V), and GND. The signal pin controls ON/OFF. In our project, we turn it ON at startup and leave it on — it acts as a visual pointer so you can see exactly what object you are measuring.

 **Important:** The HW-493 is a **transmitter only**. It cannot measure distance by itself. It just shoots a beam.

Specifications:

Spec	Value
Wavelength	650 nm (Red)
Output Power	5 mW
Operating Voltage	3V – 5V
Operating Current	< 40 mA
Class	Class 2 Laser

3. Circuit Diagram & Connections

HC-SR04 → Arduino Uno

HC-SR04 Pin	Arduino Pin	Notes
VCC	5V	Power supply
GND	GND	Common ground
TRIG	Digital Pin 9	Trigger pulse output
ECHO	Digital Pin 10	Echo pulse input

LCD I2C → Arduino Uno

LCD I2C Pin	Arduino Pin	Notes
VCC	5V	Power supply
GND	GND	Common ground
SDA	Analog Pin A4	I2C data line
SCL	Analog Pin A5	I2C clock line

HW-493 Laser → Arduino Uno

Laser Pin	Arduino Pin	Notes
S (Signal)	Digital Pin 7	Controls ON/OFF
Middle pin (VCC)	5V	Power supply
– (GND)	GND	Common ground

Circuit Description

The Arduino acts as the central hub. All three peripherals share the 5V and GND rails. The HC-SR04 connects to digital pins 9 and 10. The LCD communicates over the I2C bus (A4/A5) — a shared bus where multiple devices can exist on the same two wires. The laser module connects to digital pin 7 and stays ON during operation.

Power is supplied via USB. A **5V 1A wall adapter** is recommended to avoid voltage drops when all components run simultaneously.

4. The Code — Explained Line by Line

4.1 Libraries & Includes

```
1  #include <Arduino.h>           // Core Arduino functions
   (pinMode, digitalWrite, etc.)
2  #include <Wire.h>              // I2C communication
   library
```


`setup()` runs once when Arduino powers on. We configure each pin as INPUT or OUTPUT, turn the laser on, and show a startup message on the LCD.

4.4 `getDistance()` — The Measurement Function

```
1  float getDistance() {
2      digitalWrite(TRIG_PIN, LOW);
3      delayMicroseconds(2);           // Clean the
    trigger pin
4      digitalWrite(TRIG_PIN, HIGH);
5      delayMicroseconds(10);         // Send 10µs
    pulse
6      digitalWrite(TRIG_PIN, LOW);
7
8      long duration = pulseIn(ECHO_PIN, HIGH, 30000); //
    Measure echo duration
9      if (duration == 0) return -1;   //
    Timeout = no object detected
10     return (duration * 0.0343) / 2.0; //
    Convert to cm
11 }
```

`pulseIn()` is a built-in Arduino function that measures how long a pin stays HIGH — that duration is the round-trip travel time of the sound wave. The timeout of 30000 µs prevents hanging if no echo returns.

4.5 `loop()` — Runs Forever

```
1  void loop() {
2      float distance = getDistance();
3
4      lcd.setCursor(0, 0);
5      lcd.print("Dist:");
6
7      lcd.setCursor(0, 1);
8      if (distance < 0 || distance > 400) {
```

```

 9      lcd.print("Out of range    ");
10    } else {
11      lcd.print(distance, 1);      // 1 decimal place
12      lcd.print(" cm              "); // Trailing spaces clear
    leftover characters
13    }
14
15    delay(200); // Refresh every 200ms
16  }

```

Every 200 milliseconds, the loop measures distance and updates the LCD. Trailing spaces after "cm" are important — they overwrite leftover characters from previous longer readings.

5. How It All Works Together

Here is the complete flow from power-on to displaying a reading:

1. Arduino powers on → `setup()` runs → Laser turns on, LCD shows "Distance Meter" for 2 seconds
2. `loop()` begins → `getDistance()` is called every 200ms
3. TRIG pin sends a 10µs pulse → HC-SR04 fires 8 ultrasonic bursts at 40kHz
4. Sound waves travel through air at 343 m/s and hit an object
5. Echo bounces back → HC-SR04 raises the ECHO pin HIGH
6. Arduino measures ECHO pulse duration using `pulseIn()`
7. Duration is converted to centimetres: $(\text{duration} \times 0.0343) / 2$
8. LCD is updated with the new distance value
9. Red laser dot shows exactly what is being measured
10. Process repeats every 200ms — giving **5 readings per second**

Why 200ms Refresh Rate?

The HC-SR04 needs time between measurements to avoid its own echo interfering with the next reading. 200ms gives smooth updates without interference. Going faster than ~50ms can cause ghost readings.

Why Does the Arduino Reset When the Sensor is Connected?

This is caused by insufficient current. When the HC-SR04 fires, it draws a current spike. If powered from a laptop USB port (limited to 500mA shared across all components), voltage drops momentarily causing an Arduino reset. **Fix: use a dedicated 5V 1A wall adapter.**

6. Limitations & Things to Know

Limitation	Explanation
Soft surfaces absorb sound	Fabric, foam, and carpet absorb ultrasonic waves instead of reflecting them — readings may be inaccurate
Angle matters	Objects must be roughly perpendicular to the sensor. Angled surfaces deflect sound away and may return no echo
Minimum range is 2cm	Too close and the echo returns before the sensor is ready to receive
Temperature affects accuracy	Sound speed changes with temperature. At room temp (~25°C) the formula is accurate enough
Laser is only a pointer	The HW-493 cannot measure distance — it only shows what is being aimed at

7. Summary

This project demonstrates core embedded systems concepts: **digital I/O**, **pulse timing**, **I2C communication**, and **real-time display updates**.

The HC-SR04 provides reliable distance measurements from 2cm to 4m using ultrasonic echo timing. The Arduino processes timing data using the speed of sound formula and displays results on the I2C LCD. The HW-493 laser provides a visual indicator for the measurement target.

The project successfully integrates three peripherals over different communication protocols (digital GPIO for the sensor, I2C for the display)

and demonstrates how microcontrollers orchestrate multiple hardware components in real time.